

GUÍA DE LABORATORIO 4

Pruebas End to End – Notes API

Proyecto: notes-api-db

Materia: Verificación y Validación de Software (INF780)

Nivel: Universitario | Duración: 3 horas

Herramientas: NestJS · Jest · TypeScript · PostgreSQL

2026

GUÍA DE LABORATORIO N° 4

Pruebas End-to-End (E2E) — Notes API

Dato	Detalle
Materia	INF780 — Verificación y Validación de Software
Proyecto	notes-api-db (NestJS + TypeORM + PostgreSQL)
Repositorio	github.com/HuascarFedor/INF780-NotesApiDb
Tag de versión	v4.0
Archivo principal	test/notes.e2e-spec.ts
Duración estimada	3 horas
Prerrequisitos	Guía 2 (pruebas unitarias) y Guía 3 (pruebas de integración)

1. Objetivos de Aprendizaje

- Comprender qué son las pruebas end-to-end y su ubicación en la pirámide de testing.
- Configurar un entorno de pruebas E2E con una base de datos PostgreSQL real.
- Utilizar la librería supertest para simular peticiones HTTP contra la API.
- Verificar el ciclo de vida completo de un recurso REST: crear, listar, obtener, actualizar y eliminar.
- Validar que los pipes globales (ValidationPipe, ParseUUIDPipe), filtros e interceptores funcionan correctamente en conjunto.
- Interpretar códigos de estado HTTP (201, 200, 204, 400, 404, 422) como indicadores de comportamiento correcto.

2. Marco Teórico

2.1. ¿Qué son las pruebas End-to-End?

Las pruebas end-to-end (E2E) verifican el comportamiento del sistema completo, desde la recepción de una petición HTTP hasta la respuesta final, pasando por todas las capas intermedias: controladores, pipes, interceptores, servicios y acceso a base de datos. A diferencia de las pruebas unitarias (que aíslan una unidad con mocks) y las pruebas de integración (que verifican la colaboración entre componentes), las pruebas E2E ejecutan el flujo real sin simulaciones.

Característica	Prueba Unitaria / Integración	Prueba E2E
Base de datos	Mock (repositorio simulado)	PostgreSQL real (synchronize: true)
Servidor HTTP	No se levanta / parcial	Se levanta completo con app.init()

Característica	Prueba Unitaria / Integración	Prueba E2E
Pipes y filtros	No participan / parcial	Participan todos (ValidationPipe, HttpExceptionFilter)
Peticiones	Llamadas directas al servicio/controlador	HTTP real vía supertest
Velocidad	Rápidas (ms)	Más lentas (requieren I/O de red y BD)
Confianza	Aislada por componente	Alta: valida el sistema integrado

2.2. Ubicación en la Pirámide de Testing

La pirámide de testing propone tener **muchas pruebas unitarias** en la base (rápidas y baratas), una cantidad **moderada de pruebas de integración** en el medio, y **pocas pruebas E2E** en la cima (lentas pero de alta confianza). En este proyecto hemos seguido exactamente esa secuencia: Guía 2 → unitarias, Guía 3 → integración, Guía 4 → E2E.

2.3. supertest: Peticiones HTTP simuladas

La librería `supertest` permite enviar peticiones HTTP reales contra una instancia de la aplicación NestJS sin necesidad de levantar un servidor en un puerto externo. Se integra directamente con el `HttpServer` interno de NestJS, lo que permite verificar rutas, códigos de estado, headers y cuerpos de respuesta.

Sintaxis básica:

```
const response = await request(app.getHttpServer())
  .post('/notes') // Método y ruta
  .send({ title: 'Test', content: 'Hi' }) // Body (para POST/PATCH)
  .expect(201); // Código esperado

expect(response.body.data.title).toBe('Test');
```

2.4. Configuración del Módulo de Testing E2E

A diferencia de las pruebas unitarias donde se usan mocks, en E2E se configura el módulo de testing con los **módulos reales** de la aplicación. Esto incluye:

Componente	Rol en las pruebas E2E
ConfigModule.forRoot()	Carga variables de entorno desde <code>.env.test</code> con validación Joi
TypeOrmModule.forRootAsync()	Conecta a PostgreSQL real usando las variables de configuración

Componente	Rol en las pruebas E2E
NotesModule	Importa el módulo completo (controlador + servicio + repositorio)
ValidationPipe	Valida DTOs, rechaza propiedades extra, retorna 422 en errores
HttpExceptionFilter	Formatea las respuestas de error de forma consistente
ResponseInterceptor	Envuelve las respuestas exitosas en { statusCode, data }

2.5. Flujo de una prueba E2E

Cada prueba E2E sigue este flujo:

1	beforeAll Compila el módulo de testing, configura pipes/filtros/interceptores globales y levanta la app con app.init().
2	Ejecución de pruebas Cada it() envía una petición HTTP con supertest, verifica el código de estado y el cuerpo de la respuesta.
3	Estado compartido Las pruebas pueden compartir estado (como createdNoteld) para simular flujos completos: crear → leer → actualizar → eliminar.
4	afterAll Cierra la conexión a la base de datos y detiene la aplicación con app.close().

2.6. Códigos de Estado HTTP en las Pruebas

Las pruebas E2E verifican que la API responda con los códigos correctos según la situación:

Código	Endpoint(s)	Significado
200 OK	GET /notes, GET /notes/:id, PATCH /notes/:id	Operación exitosa con datos en el body
201 Created	POST /notes	Recurso creado exitosamente
204 No Content	DELETE /notes/:id	Eliminación exitosa, sin body
400 Bad Request	GET /notes/not-a-uuid	UUID inválido (ParseUUIDPipe)
404 Not Found	GET, PATCH, DELETE con UUID inexistente	Recurso no encontrado
422 Unprocessable Entity	POST /notes sin title o con title > 200 chars	Validación de DTO fallida

3. Desarrollo

En esta sección construiremos el archivo de pruebas E2E paso a paso. El archivo final se ubica en `test/notes.e2e-spec.ts` y verifica el ciclo CRUD completo de la API de notas.

Paso 1: Preparación del entorno

Antes de escribir las pruebas, necesitamos asegurar que las dependencias estén instaladas y el archivo `.env.test` esté configurado:

```
# Instalar supertest (si no está instalado)
npm install --save-dev supertest @types/supertest
```

Crear el archivo `.env.test` en la raíz del proyecto con las credenciales de la base de datos de pruebas:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=notes_test_user
DB_PASSWORD=tu_password
DB_NAME=notes_test_db
NODE_ENV=test
PORT=3001
```

Importante: Se recomienda usar una base de datos exclusiva para pruebas (notes_test_db) y no la misma de desarrollo, ya que las pruebas E2E pueden crear y eliminar datos reales.

Paso 2: Imports y estructura del archivo

Creemos el archivo `test/notes.e2e-spec.ts`. NestJS ya incluye la carpeta `test/` con una configuración de Jest específica para E2E (`test/jest-e2e.json`). Comenzamos con los imports necesarios:

```
import { INestApplication, ValidationPipe } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { Test, TestingModule } from '@nestjs/testing';
import { TypeOrmModule } from '@nestjs/typeorm';
import * as Joi from 'joi';
import request from 'supertest';
import { HttpExceptionFilter } from '../src/common/filters/http-exception.filter';
import { ResponseInterceptor } from
'../src/common/interceptors/response.interceptor';
```

```
import { Note } from '../src/notes/entities/note.entity';
import { NotesModule } from '../src/notes/notes.module';
```

Observa que importamos componentes tanto del framework (`@nestjs/common`, `@nestjs/testing`) como de nuestra aplicación (`HttpExceptionFilter`, `ResponseInterceptor`, `Note`, `NotesModule`). También importamos `Joi` para la validación del esquema de variables de entorno y `supertest` para las peticiones HTTP.

Paso 3: Configuración del módulo de testing

Dentro del describe principal, declaramos las variables compartidas y configuramos el módulo en `beforeAll`:

```
describe('Notes (e2e)', () => {
  let app: INestApplication;
  let createdNoteId: string;

  beforeAll(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [
        ConfigModule.forRoot({
          isGlobal: true,
          envFilePath: ['.env.test', '.env'],
          validationSchema: Joi.object({
            DB_HOST: Joi.string().default('localhost'),
            DB_PORT: Joi.number().default(5432),
            DB_USER: Joi.string().required(),
            DB_PASSWORD: Joi.string().required(),
            DB_NAME: Joi.string().required(),
            NODE_ENV: Joi.string()
              .valid('development', 'production', 'test')
              .default('test'),
          }),
        }),
      ],
    }).compile();

    app = moduleFixture.createNestApplication();
    app.useFactory({
      type: 'postgres',
      host: config.get<string>('DB_HOST', 'localhost'),
      port: config.get<number>('DB_PORT', 5432),
      username: config.get<string>('DB_USER'),
      password: config.get<string>('DB_PASSWORD'),
      database: config.get<string>('DB_NAME'),
      entities: [Note],
      synchronize: true,
      dropSchema: false,
```

```
    }),  
  },  
  NotesModule,  
],  
}).compile();
```

*Nota: synchronize: true permite que TypeORM cree/actualice las tablas automáticamente.
dropSchema: false evita que se eliminen los datos existentes entre ejecuciones. En un entorno de CI/CD podría usarse dropSchema: true para garantizar un estado limpio.*

Paso 4: Configuración de pipes, filtros e interceptores

Después de compilar el módulo, creamos la instancia de la aplicación y registramos los mismos componentes globales que usa la aplicación real. Esto es **crítico** para que las pruebas E2E reflejen el comportamiento exacto de producción:

```
app = moduleFixture.createNestApplication();  
  
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,           // Elimina propiedades no declaradas en el DTO  
    forbidNonWhitelisted: true, // Rechaza propiedades extra con error  
    transform: true,          // Transforma tipos automáticamente  
    errorHttpStatusCode: 422,  // Usa 422 en lugar del 400 por defecto  
  }),  
);  
app.useGlobalFilters(new HttpExceptionFilter());  
app.useGlobalInterceptors(new ResponseInterceptor());  
  
await app.init();  
});  
  
afterAll(async () => {  
  await app.close();  
});
```

El `ValidationPipe` con `errorHttpStatusCode: 422` es el responsable de que los errores de validación retornen 422 (Unprocessable Entity) en lugar del 400 por defecto de NestJS. El `HttpExceptionFilter` formatea los errores, y el `ResponseInterceptor` envuelve las respuestas exitosas en la estructura `{ statusCode, data }`.

Paso 5: Pruebas POST /notes

El primer grupo de pruebas verifica la creación de notas, incluyendo los casos de éxito y los errores de validación:

```
describe('POST /notes', () => {
  it('should create a note and return 201 with correct body', async () => {
    const response = await request(app.getHttpServer())
      .post('/notes')
      .send({ title: 'E2E Test Note', content: 'This is e2e test content' })
      .expect(201);

    expect(response.body.statusCode).toBe(201);
    expect(response.body.data).toMatchObject({
      title: 'E2E Test Note',
      content: 'This is e2e test content',
      isArchived: false,
    });
    expect(response.body.data.id).toBeDefined();
    expect(typeof response.body.data.id).toBe('string');

    createdNoteId = response.body.data.id;
  });

  it('should return 422 when title is missing', async () => {
    await request(app.getHttpServer())
      .post('/notes')
      .send({ content: 'Only content, no title' })
      .expect(422);
  });

  it('should return 422 when title exceeds 200 characters', async () => {
    await request(app.getHttpServer())
      .post('/notes')
      .send({ title: 'a'.repeat(201), content: 'Some content' })
      .expect(422);
  });
});
```

Observa cómo el primer test almacena el `createdNoteId` para reutilizarlo en las pruebas siguientes. Este patrón de **estado compartido** es común en pruebas E2E, donde simulamos un flujo de usuario completo.

Patrón clave: En pruebas E2E, los tests dentro de un describe se ejecutan en orden secuencial. Esto permite que un test cree un recurso y los siguientes lo consulten, actualicen y eliminen, simulando un flujo de usuario real.

Paso 6: Pruebas GET /notes

Verificamos que el endpoint de listado funciona correctamente, incluyendo el filtro por estado de archivado:

```
describe('GET /notes', () => {
  it('should return 200 with an array containing the created note', async () => {
    {
      const response = await request(app.getHttpServer())
        .get('/notes')
        .expect(200);

      expect(response.body.statusCode).toBe(200);
      expect(Array.isArray(response.body.data)).toBe(true);

      const found = response.body.data.find(
        (n: Note) => n.id === createdNoteId
      );
      expect(found).toBeDefined();
      expect(found.title).toBe('E2E Test Note');
    });

    it('should filter only archived notes when ?archived=true', async () => {
      const response = await request(app.getHttpServer())
        .get('/notes?archived=true')
        .expect(200);

      const hasNonArchived = response.body.data.some(
        (n: Note) => !n.isArchived
      );
      expect(hasNonArchived).toBe(false);
    });

    it('should filter only non-archived notes when ?archived=false', async () => {
      const response = await request(app.getHttpServer())
        .get('/notes?archived=false')
        .expect(200);

      const hasArchived = response.body.data.some(
        (n: Note) => n.isArchived
      );
      expect(hasArchived).toBe(false);
    });
  });
});
```

El test de listado busca la nota creada en el paso anterior usando `Array.find()`. Los tests de filtrado verifican que el query parameter `?archived` funcione correctamente, asegurando que no se incluyan notas con el estado opuesto.

Paso 7: Pruebas GET /notes/:id

Verificamos la obtención de una nota individual, incluyendo los casos de error:

```
describe('GET /notes/:id', () => {
  it('should return 200 with the correct note data', async () => {
    const response = await request(app.getHttpServer())
      .get(`/notes/${createdNoteId}`)
      .expect(200);

    expect(response.body.data.id).toBe(createdNoteId);
    expect(response.body.data.title).toBe('E2E Test Note');
    expect(response.body.data.content).toBe('This is e2e test content');
  });

  it('should return 404 when note does not exist', async () => {
    await request(app.getHttpServer())
      .get('/notes/00000000-0000-0000-0000-000000000000')
      .expect(404);
  });

  it('should return 400 when UUID is invalid', async () => {
    await request(app.getHttpServer())
      .get('/notes/not-a-uuid')
      .expect(400);
  });
});
```

Este grupo prueba tres escenarios fundamentales: el caso exitoso, el caso de recurso no encontrado (404) usando un UUID válido pero inexistente (00000000-0000-0000-0000-000000000000), y el caso de UUID inválido (400) que es interceptado por el `ParseUUIDPipe` antes de llegar al servicio.

Paso 8: Pruebas PATCH /notes/:id

Las pruebas de actualización verifican tanto la modificación parcial de campos como el archivado:

```
describe('PATCH /notes/:id', () => {
  it('should return 200 with the updated field', async () => {
    const response = await request(app.getHttpServer())
      .patch(`/notes/${createdNoteId}`)
      .send({ title: 'Updated E2E Title' })
      .expect(200);

    expect(response.body.data.id).toBe(createdNoteId);
    expect(response.body.data.title).toBe('Updated E2E Title');
    expect(response.body.data.content).toBe('This is e2e test content');
  });
});
```

```
it('should return 200 when archiving a note', async () => {
  const response = await request(app.getHttpServer())
    .patch(`/notes/${createdNoteId}`)
    .send({ isArchived: true })
    .expect(200);

  expect(response.body.data.isArchived).toBe(true);
});

it('should return 404 when note does not exist', async () => {
  await request(app.getHttpServer())
    .patch('/notes/00000000-0000-0000-0000-000000000000')
    .send({ title: 'Ghost note' })
    .expect(404);
});
});
```

El primer test demuestra la **actualización parcial**: solo se envía el campo `title`, y se verifica que `content` permanece sin cambios. El segundo test archiva la nota cambiando `isArchived` a `true`, lo cual también afecta las pruebas de filtrado del GET anterior.

Paso 9: Pruebas DELETE /notes/:id

Las pruebas de eliminación verifican el código 204 (sin contenido) y la confirmación posterior:

```
describe('DELETE /notes/:id', () => {
  it('should return 204 with no body', async () => {
    const response = await request(app.getHttpServer())
      .delete(`/notes/${createdNoteId}`)
      .expect(204);

    expect(response.body).toEqual({});
  });

  it('should return 404 when note does not exist', async () => {
    await request(app.getHttpServer())
      .delete('/notes/00000000-0000-0000-0000-000000000000')
      .expect(404);
  });
});
```

Paso 10: Verificación post-eliminación

El último grupo de pruebas confirma que la nota eliminada ya no es accesible:

```
describe('GET /notes/:id (after deletion)', () => {
  it('should return 404 for the deleted note', async () => {
    await request(app.getHttpServer())
      .get(`/notes/${createdNoteId}`)
      .expect(404);
  });
});
```

Este último test cierra el ciclo completo: la nota que fue creada en POST, consultada en GET, actualizada en PATCH y eliminada en DELETE, ahora retorna 404 al intentar accederla. Esto verifica que la **eliminación fue efectiva** en la base de datos.

Paso 11: Ejecución de las pruebas

Para ejecutar las pruebas E2E, usamos el script definido en `package.json`:

```
npm run test:e2e
```

La salida esperada debería mostrar todos los tests pasando:

```
PASS test/notes.e2e-spec.ts
Notes (e2e)
  POST /notes
    ✓ should create a note and return 201 with correct body
    ✓ should return 422 when title is missing
    ✓ should return 422 when title exceeds 200 characters
  GET /notes
    ✓ should return 200 with an array containing the created note
    ✓ should filter only archived notes when ?archived=true
    ✓ should filter only non-archived notes when ?archived=false
  GET /notes/:id
    ✓ should return 200 with the correct note data
    ✓ should return 404 when note does not exist
    ✓ should return 400 when UUID is invalid
  PATCH /notes/:id
    ✓ should return 200 with the updated field
    ✓ should return 200 when archiving a note
    ✓ should return 404 when note does not exist
  DELETE /notes/:id
    ✓ should return 204 with no body
    ✓ should return 404 when note does not exist
```

```
GET /notes/:id (after deletion)
```

```
✓ should return 404 for the deleted note
```

```
Test Suites: 1 passed, 1 total
```

```
Tests:      15 passed, 15 total
```

4. Resumen de Pruebas

La siguiente tabla resume las 15 pruebas E2E implementadas:

Endpoint	Prueba	Código
POST /notes	Crear nota con datos válidos	201
POST /notes	Enviar sin título	422
POST /notes	Título excede 200 caracteres	422
GET /notes	Listar notas (incluye la creada)	200
GET /notes?archived=true	Filtrar solo archivadas	200
GET /notes?archived=false	Filtrar solo no archivadas	200
GET /notes/:id	Obtener nota por ID válido	200
GET /notes/:id	UUID válido pero inexistente	404
GET /notes/:id	UUID inválido	400
PATCH /notes/:id	Actualizar título	200
PATCH /notes/:id	Archivar nota	200
PATCH /notes/:id	Nota inexistente	404
DELETE /notes/:id	Eliminar nota existente	204
DELETE /notes/:id	Nota inexistente	404
GET /notes/:id	Verificar eliminación	404

5. Rúbrica de Evaluación

Criterio	Puntaje
Configuración correcta del módulo de testing E2E (ConfigModule, TypeOrmModule, NotesModule)	15
Registro de pipes globales (ValidationPipe con errorHttpStatusCode: 422)	10
Registro de filtros e interceptores globales	5
Pruebas POST /notes: creación exitosa y validaciones (3 tests)	20
Pruebas GET /notes: listado y filtrado por archived (3 tests)	15
Pruebas GET /notes/:id: caso exitoso, 404 y 400 (3 tests)	10
Pruebas PATCH /notes/:id: actualización parcial, archivado y 404 (3 tests)	10
Pruebas DELETE /notes/:id y verificación post-eliminación (3 tests)	10
Todas las pruebas pasan sin errores y la conexión se cierra correctamente	5
TOTAL	100